



ETSI GS QKD 004 V2.1.1 (2020-08)

그룹 규격(GROUP SPECIFICATION)

양자 키 분배(QKD); 애플리케이션 인터페이스

(Quantum Key Distribution (QKD); Application Interface)

면책 조항(Disclaimer)

이 문서는 양자 키 분배(QKD) ETSI 산업 규격 그룹(ISG)이 작성하고 승인하였으며, 이 ISG 에 참여한 회원들의 견해를 나타낸다. 이 문서가 반드시 ETSI 전체 회원의 견해를 나타내는 것은 아니다.

참조 번호(Reference)

RGS/QKD-004ed2_ApplIntf

키워드(Keywords)

API, quantum cryptography, quantum key distribution, security, use case

ETSI

650 Route des Lucioles

F-06921 Sophia Antipolis Cedex - FRANCE

Tel.: +33 4 92 94 42 00 Fax: +33 4 93 65 47 16

Siret N° 348 623 562 00017 - NAF 742 C

그라스(06) 지방청에 등록된 비영리 협회 N° 7803/88

중요 고지(Important notice)

이 문서는 <http://www.etsi.org/standards-search> 에서 내려받을 수 있다.

이 문서는 전자판 및/또는 인쇄판으로 제공될 수 있다. 이 문서의 전자판 및/또는 인쇄판의 내용은 ETSI 의 사전 서면 승인 없이 수정해서는 안 된다. 이러한 판본 및/또는 인쇄본 사이에 내용상 차이가 존재하거나 존재한다고 인식되는 경우, 우선하는 ETSI 성과물은 www.etsi.org/deliver 에서 PDF 형식으로 공개된 판본이다.

이 문서의 사용자는 이 문서가 개정되거나 상태가 변경될 수 있음에 유의해야 한다. 이 문서 및 기타 ETSI 문서의 현재 상태에 관한 정보는 <https://portal.etsi.org/TB/ETSIDeliverableStatus.aspx> 에서 확인할 수 있다.

이 문서에서 오류를 발견한 경우, 다음 서비스 중 하나로 의견을 보내기 바란다 :
<https://portal.etsi.org/People/CommitteeSupportStaff.aspx>

저작권 고지(Copyright Notification)

ETSI 의 서면 허가에 의해 승인된 경우를 제외하고, 복사 및 마이크로필름을 포함하여 전자적 또는 기계적인 어떠한 형태나 수단으로도 이 문서의 어떠한 부분도 복제하거나 이용할 수 없다. PDF 판본의 내용은 ETSI 의 서면 승인 없이 수정해서는 안 된다. 저작권 및 앞의 제한은 모든 매체에서의 복제에 적용된다.

© ETSI 2020. 모든 권리 보유.

DECT™, PLUGTESTS™, UMTS™ 및 ETSI 로고는 ETSI 가 회원의 이익을 위해 등록한 상표이다. 3GPP™와 LTE™는 ETSI 가 회원 및 3GPP 조직 파트너의 이익을 위해 등록한 상표이다. oneM2M™ 로고는 ETSI 가 회원 및 oneM2M 파트너의 이익을 위해 등록한 상표이다. GSM®과 GSM 로고는 GSM 협회가 등록하고 소유한 상표이다.

목 차

지식재산권(IPR)

머리말

조동사 용어

1 적용 범위

2 참고 문헌

2.1 규범적 참고 문헌

2.2 정보성 참고 문헌

3 용어, 기호 및 약어의 정의

3.1 용어

3.2 기호

3.3 약어

4 서론

5 QKD 애플리케이션 인터페이스 규격 설명

6 QKD 애플리케이션 인터페이스 API 규격

6.1 일반

6.2 QKD 애플리케이션 인터페이스의 시퀀스 다이어그램

6.2.1 일반

6.2.2 경우 1: 단일 링크 시나리오에서 KSID 가 정의되지 않은 경우

6.2.3 경우 2: 단일 링크 시나리오에서 KSID 가 정의되지 않고 키 획득 호출이 실패한 경우

6.2.4 경우 3: 단일 링크 시나리오에서 KSID 가 사전 정의된 경우

6.2.5 경우 4: 단일 링크 시나리오에서 KSID 가 사전 정의되고 키 획득 호출이 실패한 경우

6.2.6 경우 5: QKD 네트워크에서의 애플리케이션 탐색

부속서 A (공란)

부속서 B (정보성) 기존 키 관리 시스템

부속서 C (정보성) 이 API와 ETSI GS QKD 014 "REST 기반 키 전달 API의 프로토콜 및 데이터 형식"의 관계

부속서 D (정보성) 참고 자료

부속서 E (정보성) 변경 이력

이력

지식재산권(Intellectual Property Rights)

필수 특허

규범적 성과물에 필수적이거나 필수적일 가능성이 있는 지식재산권(IPR)은 ETSI에 신고되었을 수 있다. 이러한 필수 IPR에 관한 정보가 있는 경우, 해당 정보는 ETSI 회원과 비회원 모두에게 공개되며 ETSI 사무국에서 제공하는 ETSI SR 000 314 "Intellectual Property Rights (IPRs); Essential, or potentially Essential, IPRs notified to ETSI in respect of ETSI standards"에서 확인할 수 있다. 최신 갱신 사항은 ETSI 웹 서버(<https://ipr.etsi.org/>)에서 제공된다.

ETSI IPR 정책에 따라, ETSI는 IPR 검색을 포함한 어떠한 조사도 수행하지 않았다. ETSI SR 000 314(또는 ETSI 웹 서버의 갱신본)에 참조되지 않은 다른 IPR로서 이 문서에 필수적이거나, 필수적일 수 있거나, 향후 필수적이 될 수 있는 IPR이 존재하지 않는다는 점은 보장할 수 없다.

상표

이 문서에는 소유자가 주장하거나 등록한 상표 및/또는 상호가 포함될 수 있다. ETSI는 ETSI의 소유로 표시된 것을 제외하고는 이들에 대한 소유권을 주장하지 않으며, 상표 및/또는 상호를 사용하거나 복제할 권리를 부여하지 않는다. 이 문서에서 해당 상표를 언급하는 것이 그 상표와 관련된 제품, 서비스 또는 조직을 ETSI가 보증함을 의미하지는 않는다.

머리말(Foreword)

이 그룹 규격(GS)은 ETSI 산업 규격 그룹(ISG) 양자 키 분배(QKD)가 작성하였다.

조동사 용어(Modal verbs terminology)

이 문서에서 "shall", "shall not", "should", "should not", "may", "need not", "will", "will not", "can", "cannot"은 ETSI 작성 규칙(ETSI Drafting Rules)의 3.2 절(조항 표현을 위한 동사 형태)에 기술된 대로 해석한다.

"must"와 "must not"은 직접 인용에 사용되는 경우를 제외하고 ETSI 성과물에서 사용이 허용되지 않는다.

1 적용 범위(Scope)

이 문서는 QKD 키 관리자와 애플리케이션 사이의 응용 프로그래밍 인터페이스(API)를 규정하는 것을 목적으로 한다. QKD 키 관리자의 기능은 QKD 프로토콜 구현이 생성한 안전한 키를 관리하고, 이 API 를 통해 통신 종단점의 연관된 애플리케이션에 동일한 키 집합을 전달하는 것이다.

2 참고 문헌(References)

2.1 규범적 참고 문헌

참고 문헌은 특정 판본을 지정하거나(발행일 및/또는 판 번호나 버전 번호로 식별) 지정하지 않는다. 특정 판본을 지정한 경우에는 인용된 판본만 적용된다. 특정 판본을 지정하지 않은 경우에는 해당 문서의 최신판(모든 개정 포함)이 적용된다.

예상 위치에서 공개적으로 이용할 수 없는 참조 문서는 <https://docbox.etsi.org/Reference/> 에서 찾을 수 있다.

주 - 이 절에 포함된 하이퍼링크는 발행 시점에는 유효하였으나, ETSI 는 그 장기적 유효성을 보장할 수 없다.

다음 참조 문서는 이 문서의 적용에 필요하다.

[1] IANA Media Type registry list of Directories of Content Types and Subtypes.

주 - <http://www.iana.org/assignments/media-types/> 에서 이용할 수 있다.

2.2 정보성 참고 문헌

참고 문헌은 특정 판본을 지정하거나(발행일 및/또는 판 번호나 버전 번호로 식별) 지정하지 않는다. 특정 판본을 지정한 경우에는 인용된 판본만 적용된다. 특정 판본을 지정하지 않은 경우에는 해당 문서의 최신판(모든 개정 포함)이 적용된다.

주 - 이 절에 포함된 하이퍼링크는 발행 시점에는 유효하였으나, ETSI 는 그 장기적 유효성을 보장할 수 없다.

다음 참조 문서는 이 문서의 적용에 필수적이지는 않으나, 특정 주제 영역과 관련하여 사용자에게 도움이 된다.

[i.1] OASIS Standard (22 November 2017): "Key Management Interoperability Protocol Specification Version 1.4". Editor by Tony Cox.

[i.2] ETSI GS QKD 014 (V1.1.1): "Quantum Key Distribution (QKD); Protocol and data format of REST-based key delivery API".

주 - [i.1]은 <http://docs.oasis-open.org/kmip/spec/v1.4/os/kmip-spec-v1.4-os.html> 에서 이용할 수 있다.

3 용어, 기호 및 약어의 정의

3.1 용어(Terms)

이 문서에서는 다음 용어를 적용한다.

응용 프로그래밍 인터페이스 (API, Application Programming Interface): 다른 소프트웨어 프로그램과 상호작용할 수 있도록 소프트웨어 프로그램이 구현하는 인터페이스

키 관리 상호운용성 프로토콜(KMIP, Key Management Interoperability Protocol): 기업 키 관리 시스템과 암호화 시스템 간의 통신을 위한 프로토콜

주 - KMIP 은 OASIS 이니셔티브가 주관한다.

키 관리 계층(key management layer): 둘 이상의 네트워크 노드에 물리적으로 분산된 키 관리자를 포함하는 계층 모델에서의 추상화

주 - 키 관리 계층은 QKD 모듈과 여러 애플리케이션 사이에 위치한다. 키의 동기화 및 삭제 등과 애플리케이션으로의 키 전달을 관리한다.

링크 암호화기(link encryptor): 링크 암호화, 즉 데이터 링크 수준에서 두 피어 간의 정보를 암호화/복호화하는 통신 보안 과정을 수행하는 장치

구조화 정보 표준 발전 기구(OASIS, Organization for the Advancement of Structured Information Standards): KMIP 을 포함하여 전자상거래 및 웹 서비스 표준의 개발, 수렴 및 채택을 추진하는 국제 컨소시엄

QKD 애플리케이션 인터페이스(QKD application interface): QKD 키 관리자와 하나 이상의 애플리케이션 사이의 인터페이스

QKD 링크(QKD link): 한 쌍의 QKD 모듈을 연결하여 QKD 를 수행할 수 있게 하는 능동 및/또는 수동 구성 요소의 집합으로, 실행되는 하나 이상의 QKD 프로토콜 중 어느 것에서도 수립된 대칭 키의 안전성이 링크 구성 요소에 의존하지 않는 것

QKD 모듈(QKD module): 정의된 암호화 경계 내에 포함된 하드웨어, 소프트웨어 및/또는 펌웨어 구성 요소의 집합으로, 하나 이상의 QKD 프로토콜의 일부를 구현하여 적어도 하나의 다른 QKD 모듈과 대칭 키를 안전하게 수립할 수 있는 것

QKD 프로토콜(QKD protocol): 양자 얽힘 또는 전송된 양자 상태를 완벽하게 복제하는 것이 불가능하다는 성질에 기반한 안전성으로 원격 당사자 간에 대칭 키를 수립하기 위해, QKD 모듈이 수행해야 하는 양자 상태 전송을 포함한 일련의 단계

서비스 품질(QoS, Quality of Service): 서로 다른 애플리케이션이나 사용자의 우선순위를 정하기 위한 관리 정책이 적용된 후, 애플리케이션에 제공되는 서비스의 전반적인 성능에 대한 설명 또는 측도

동기화(synchronization): 두 키 관리자의 대칭 키가 동일하도록 보장하는 기능

전송 계층 보안(TLS, Transport Layer Security): 기밀성을 위해 대칭 암호를, 메시지 신뢰성을 위해 키가 적용된 메시지 인증 코드를 사용하여, 전송 계층 상위의 네트워크 연결 세그먼트를 암호화하는 데 사용되는 암호 프로토콜

3.2 기호(Symbols)

공란(Void).

3.3 약어(Abbreviations)

이 문서에서는 다음 약어를 적용한다.

AAA	Authentication, Authorization and Accounting
API	Application Programming Interface
APPA	Application at host A
APPB	Application at host B
APPZ	Application at host Z
CORBA	Common Object Request Broker Architecture
HTTPS	Hypertext Transfer Protocol Secure
IANA	Internet Assigned Numbers Authority
ITS	Information Theoretical Secure
JSON	JavaScript Object Notation
KIMP	Key Management Interoperability Protocol
KM	Key Manager
KMIP	Key Management Interoperability Protocol
KSID	Key stream ID
OASIS	Organization for the Advancement of Structured Information Standards
PAP	Password Authentication Protocol
QKD	Quantum Key Distribution
QKDA	QKD key management layer peer at host A
QKDB	QKD key management layer peer at host B
QoS	Quality of Service
REST	Representational State Transfer
SDN	Software Defined Networking

SSL	Secure Socket Layer
TLS	Transport Layer Security
TTL	Time To Live
URI	Uniform Resource Identifier

역자 주 - 원문의 약어 목록에는 "KIMP"와 "KMIP"이 모두 같은 뜻으로 실려 있다. 원문 표기를 그대로 두었으며, KIMP 는 오타로 보인다.

4 서론(Introduction)

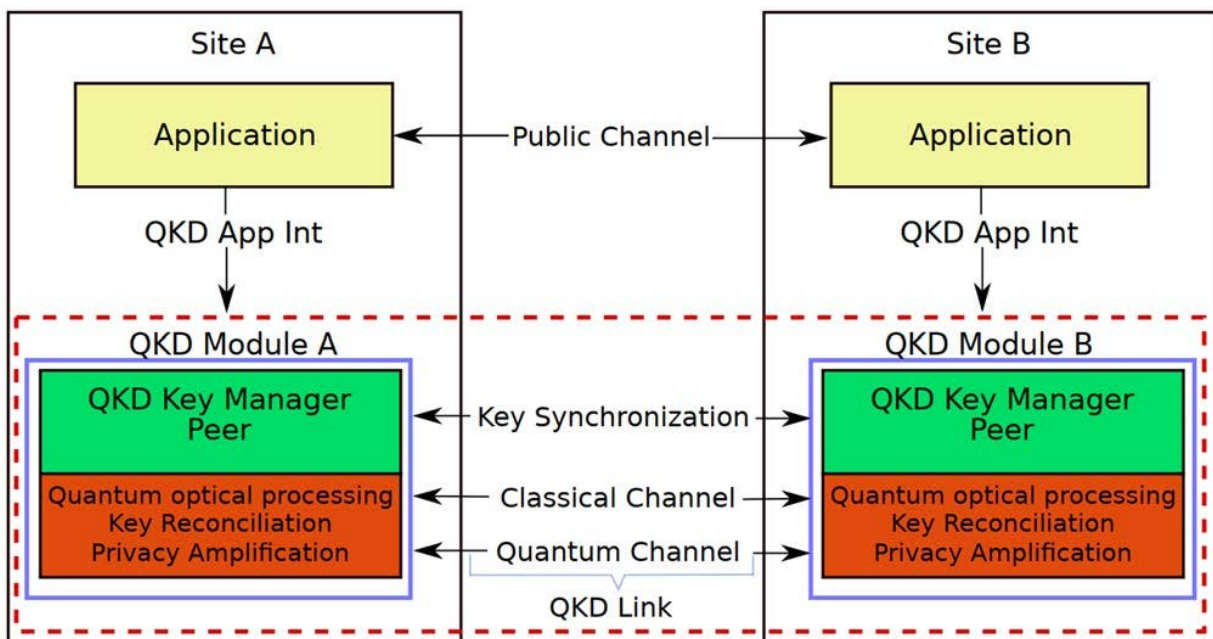
이 문서는 QKD 키 관리자(KM)와 애플리케이션 사이의 응용 프로그래밍 인터페이스(API)를 규정하는 것을 목적으로 한다. QKD KM의 기능은 QKD 프로토콜 구현이 생성한 안전한 키를 관리하고, 요청에 따라 이 API를 통해 통신 종단점의 피어 애플리케이션에 동일한 키 집합을 전달하는 것이다. 키 관리자는 여러 수준으로 존재할 수 있으며(예: 하나는 QKD 링크 수준, 다른 하나는 QKD 네트워크 수준), 이하의 절에서 규정하는 API는 모든 QKD 키 관리자 수준에서 구현될 수 있고, 이러한 키 교환은 사용자의 로컬 보안 경계 내에서 이루어진다는 점에 유의해야 한다.

제조사자는 QKD 키 관리자가 제공되는 곳이라면 어디에나 이 API를 구현할 수 있다. 제조사는 적절하다고 판단하는 추가 API와 확장 기능을 제공할 수 있으나, 이러한 추가 사항은 다른 벤더가 제공하는 버전과 호환되지 않을 수 있음을 인지해야 한다. 또한 ETSI GS QKD 014 [i.2]에 규정된 REST API와 같이, 이 기본 API 위에 상위 수준의 API를 구축할 수 있다.

5 QKD 애플리케이션 인터페이스 규격 설명

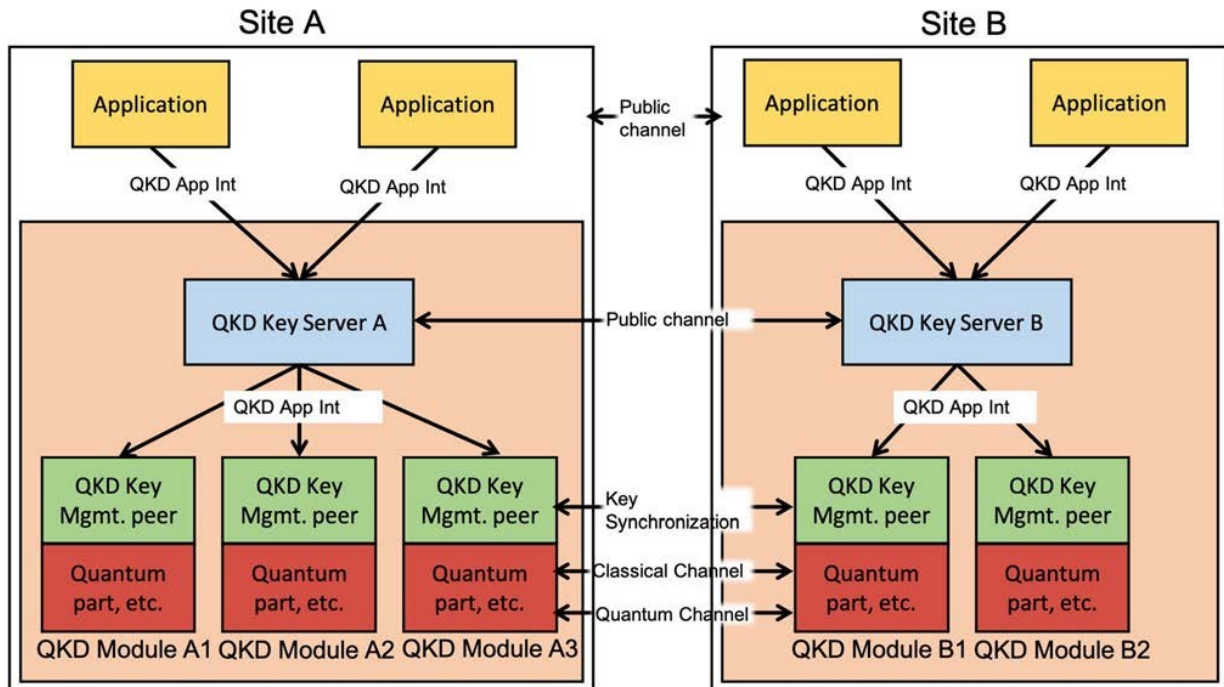
이 문서는 키 관리자가 여러 수준으로 존재할 수 있는 경우(예: 하나는 QKD 링크 수준, 다른 하나는 QKD 네트워크 수준)를 포괄하며((그림 1)과 (그림 2) 참조), 이하의 절에서 규정하는 API는 모든 QKD 키 관리자 수준에 적용될 수 있고, 이러한 교환은 사이트(site)라고 부르는 사용자의 로컬 보안 경계 내에서 이루어진다. 키 관리자가 이러한 기능을 수행하는 방법은 이 문서의 범위를 벗어나지만, 서로 다른 사이트의 키 관리자 간에는 외부 통신이 필요할 것으로 예상된다. 또한 공통의 키 연관을 알리기 위해 피어 애플리케이션 간에도 일부 통신이 필요할 수 있다.

가장 단순한 예인 (그림 1)은 사이트 A와 사이트 B에 종단점을 갖는 단일 QKD 링크를 보여준다. 각 사이트에는 하나의 애플리케이션(노란색 음영)과 하나의 QKD 모듈(파란색 상자로 둘러싸임)이 있으며, QKD 모듈은 QKD 프로토콜에서 자신이 담당하는 부분(빨간색 음영)을 구현하여 QKD 키를 생성하고, 이 키는 QKD 키 관리자 피어(녹색 음영)가 관리한다. 이 경우 단일 피어 애플리케이션은 QKD 애플리케이션 인터페이스를 사용하여 두 사이트에서 요청에 따라 동일한 안전한 키 집합을 획득한다.



(그림 1) QKD 애플리케이션 인터페이스와 피어 관계 — 사이트 A와 사이트 B는 각 사이트의 보안 경계를 나타낸다

QKD 네트워크에 있는 두 사이트의 보다 일반적인 예는 (그림 2)와 같다. 여기서는 네트워크 내 여러 사이트 중 다수의 애플리케이션을 포함하는 두 사이트를 보여준다. 사이트 A와 사이트 B는 QKD 모듈 A3과 QKD 모듈 B1 사이의 단일 QKD 링크로 연결된다. 다른 QKD 모듈들의 종단점은 표시하지 않았다. 또한 QKD 키 서버라고 부르는 네트워크 계층 키 관리자도 함께 표시하였다. QKD 키 서버의 기능은 종단점 간의 키를 관리하고 이들 종단점의 애플리케이션에 동일한 키 집합을 전달하는 것이다. 이 문서에서 규정하는 QKD 애플리케이션 인터페이스는 QKD 키 관리 피어에서 QKD 키 서버로 키를 전달하는 데에도, QKD 키 서버에서 애플리케이션으로 키를 전달하는 데에도 사용될 수 있다.



(그림 2) QKD 네트워크 내 사이트에 대한 두 수준의 QKD 애플리케이션 인터페이스와 피어 관계 — 사이트 A와 사이트 B는 각 위치의 보안 경계를 나타낸다

6 QKD 애플리케이션 인터페이스 API 규격

6.1 일반(General)

이 애플리케이션 인터페이스를 구현하는 QKD 키 관리자는 다음 API 함수를 제공해야 한다.

<표 1>

Name	설명
OPEN_CONNECT	분산된 QKD 키 관리 계층을 통해 QKD 링크 양 끝단에서 향후 사용할 키 집합에 대한 연관 (Key_stream_ID)을 예약하고, 기대되는 키 서비스 수준을 정의하는 파라미터 집합을 수립한다. 이 함수는 피어가 연결될 때까지, 또는 QoS 파라미터의 Timeout 값을 초과할 때까지 블로킹되어야 한다.
CLOSE	이 Key_stream_ID에 대해 수립된 연관을 종료한다. 연관이 종료된 후에는 이 Key_stream_ID에 더 이상 키를 할당해서는 안 된다. 링크 반대편의 시점 차이로 인해 피어 측의 이 동작은 다른 시점에 일어나므로, 사용되지 않은 키는 그때까지 보관한 후 폐기하거나 TTL(Time To Live, QoS 파라미터)이 만료될 때까지 보관해야 한다.
GET_KEY	이 Key_stream_ID에 대해 요청된 양의 키 자료를 획득한다. 각 호출은 요청된 고정량의 키를 key_buffer 파라미터로 반환하거나, 실패 사유를 나타내는 오류 메시지를 반환해야 한다. QKD 키 관리자 피어는 요청된 키와 함께 index 값을 반환하거나, 동기화를 위해 애플리케이션이 index 파라미터로 가져올 키의 위치를 지정할 수 있게 해야 한다. 키 위치는 index 값에 QoS 파라미터의 Key_chunk_size 값을 곱한 결과가 된다. Metadata_size 파라미터가 0이 아니면, 이 호출은 Metadata_buffer 파라미터로 해당 키 청크에 관한 정보를 반환한다. 0이면 메타데이터는 반환되지 않는다. Metadata_size가 0이 아니고 이 정보가 제공된 버퍼에

Name	설명
	들어가지 않으면, 호출은 키를 제공하지 않고 실패하되 Metadata_size 파라미터에 필요한 버퍼 크기를 반환하고 status 파라미터에 플래그를 설정한다. 따라서 애플리케이션은 버퍼 크기를 올바르게 설정하거나 0 으로 설정하여 키 손실 없이 호출을 다시 시도할 수 있다. 이 함수는 원하는 만큼 자주 호출할 수 있으나, QKD 키 관리자는 QoS 파라미터로 요청된 비트율 또는 시스템이 감당할 수 있는 최선의 속도로만 응답하면 된다. QKD 키 관리자는 피어와의 통신을 통해 QKD 링크 양 끝단의 키를 예약하고 동기화할 책임이 있다. 이 함수는 OPEN_CONNECT 함수를 통해 이루어진 요청에 따라 성공 또는 실패를 나타내는 status 파라미터와 함께 항상 반환된다. 이 함수의 Timeout 값은 OPEN_CONNECT() 함수에서 지정한다.

이들 함수의 구문은 다음과 같다.

```
Interface QKD{
    OPEN_CONNECT (in source, in destination, inout QoS, inout Key_stream_ID, out
status);
    GET_KEY (in Key_stream_ID, inout index, out Key_buffer, inout Metadata, out
status);
    CLOSE (in Key_stream_ID, out status);
}
```

주 - 파라미터 "Key_stream_ID"는 호출자가 처음 연결을 열면서 "NULL" 값을 전달하는 경우에는 출력 파라미터이다. 값이 "NULL"이 아니면, 그 값은 피어 측에서 최근에 수립한 "Key_stream_ID"이거나 특별히 원하거나 사전 정의된 "Key_stream_ID"로 간주된다. 그 밖의 모든 함수에서 "Key_stream_ID"는 입력이다. 해당 "Key_stream_ID"가 이미 다른 애플리케이션에서 사용 중이면 OPEN_CONNECT 함수는 오류와 함께 반환된다. 반환 시 QoS 구조체는 시스템이 제공할 수 있는 가장 근접한 QoS(최선 노력)이다. 입력 시 구조체의 특정 필드가 "NULL"이면, 시스템은 가장 적절하다고 판단하는 값을 자유롭게 제공할 수 있다. 특히 KSID 가 이미 등록되어 있다면, OPEN_CONNECT 호출의 QoS 값과 무관하게 해당 KSID 에 이전에 할당된 QoS 값으로 QoS 구조체가 채워진다.

함수 파라미터는 <표 2>에서 설명한다.

<표 2> API 함수 파라미터 설명

Name	설명	Type	비고
Key_stream_ID (KSID)	QKD 키 관리자가 애플리케이션에 제공하는, 동기화된 비트 그룹에 대한 고유 식별자	UUID_v4 16 bytes (128 bits)	키를 찾는 데 필요한 정보에 대한 참조를 포함하지만, 키 자료 자체는 포함하지 않는다. Key_stream_ID 로부터 키 자료를 유도할 수 없다. 문자 배열(char *)로 표현해야 하며, 상위 옥텟이 최하위 유효 옥텟이 된다. 다른 사이트의 다른 애플리케이션으로 전달되어 저장될 수 있다. 고유해야 하며, 두 피어는 자신의 애플리케이션 키 스트림을 참조할 때 동일한 값을 사용한다. 피어 애플리케이션 간에 사전에 합의하거나 공개 채널을 통해 서로 전달할 수 있다.
Key_buffer	현재의 키 스트림을 담는 버퍼	Array of bytes (octets)	Key buffer 는 옥텟 단위로 묶인 비트 배열로, octet[0] 의 bit[0]이 첫 번째 비트이고 octet[n]의 bit[7]이 8*n+8 번째 비트가 되도록 정렬된다.
Index	애플리케이션을 위해 예약된 키 저장소 내에서 접근할 키의 위치	32 bit unsigned integer	클라이언트/서버 동기화를 위해, index 값으로 예약된 키 저장소 내 어느 위치에 접근할지 지정할 수 있다. 실제 위치는 index 값에 QoS 파라미터의 Key_chunk_size 를 곱하여 계산된다.
Source	통일 자원 식별자(URI)로 정의되는 출발지 식별자	URI	QKD 키 관리 계층에 접속하는 출발지 애플리케이션의 식별자. 이 식별자는 URI 형식으로 구성된다. 클라이언트 /서버 방식이 정의된 경우, source 에는 서버의 URI 가 포함된다.
Destination	통일 자원 식별자(URI)로	URI	QKD 키 관리 계층에 접속하는 목적지 애플리케이션의

Name	설명	Type	비고
	정의되는 목적지 식별자		식별자. 이 식별자는 URI 형식으로 구성된다. 클라이언트/서버 방식이 정의된 경우, source 에는 클라이언트의 URI 가 포함된다.
QoS	요청하는 키 소스의 특성을 기술하는 구조체		
Key_chunk_size	애플리케이션이 요청하는 키 버퍼의 길이(바이트)	32 bit unsigned integer	요청한 양의 키를 제공할 수 없는 경우, status 파라미터로 오류를 반환해야 한다.
Max_bps	애플리케이션이 요청하는 최대 키 생성률(bps)	32 bit unsigned integer	키 관리 계층에 예상되는 최대 수요에 대한 지침을 제공하기 위한 것이다. 피어가 요청된 속도를 충족할 수 없는 경우, 요청 속도보다 낮더라도 최선을 다하거나 요청을 거부할 수 있다.
Min_bps	애플리케이션이 요구하는 최소 키 생성률(bps)	32 bit unsigned integer	애플리케이션이 보안 채널을 유지하는 데 필요한 최소 키 생성률을 QKD 키 관리 계층에 알리기 위한 것이다. 피어가 요청된 속도를 충족할 수 없으면 요청을 거부할 수 있다.
Jitter	키 전달에서 예상되는 최대 편차(bps)	32 bit unsigned integer	이 값으로 애플리케이션은 전달률의 최소 편차와 함께 키 비트 흐름의 변동을 지정할 수 있다.
Priority	요청의 우선순위	32 bit unsigned integer	QKD 키 관리 계층에 요청의 우선순위 수준에 대한 지침을 제공하기 위한 것이다. 이 정보의 처리 방법은 구체적인 구현에 맡긴다.
Timeout	호출이 중단되고 오류를 반환하기까지의 시간 (msec)	32 bit unsigned integer	이 시간이 지나도 API 함수 중 하나가 완료되지 않으면, 해당 함수는 status 파라미터에 TIMEOUT_ERROR 를 반환한다. 이 값은 밀리초 단위로 표현해야 한다.
Time to Live (TTL)	보안상의 이유로, 이 KSID 에 예약된 키를 애플리케이션 전용 키 저장소에서 삭제하기까지의 시간(초)	32 bit unsigned integer	이 값은 주어진 KSID 에 예약된 키의 수명을 설정한다. 어떤 이유로든 애플리케이션이 비정상 종료되거나 사용하지 않은 키를 남긴 경우를 정리하기 위한 것이다.
Metadata mimetype	각 GET 호출에서 KM 이 전달할 메타데이터의 mimetype	char array of size 256	이 필드는 이후 각 GET_KEY 호출에서 메타데이터의 형식을 정의한다. mimetype 값은 IANA(Internet Assigned Numbers Authority) 미디어 유형 등록부 [1]에 정의된 미디어 유형 중 하나여야 한다. 이 문서에 대한 적합성을 주장하는 KM 은 메타데이터 mimetype "application/json"을 지원해야 한다. KM 은 추가 mimetype 을 구현할 수 있다. mimetype 이 정의되지 않은 경우, 즉 metadata mimetype 의 첫 바이트가 0 이면, KM 은 이 키 스트림과 연관된 GET_KEY 호출에서 메타데이터를 보고해서는 안 된다. mimetype 이 지정되었으나 현재 KM 구현이 해당 mimetype 을 지원하지 않으면 OPEN_CONNECT 는 실패해야 하며, metadata mimetype 필드에는 KM 이 선호하는 mimetype 을 반환해야 한다. 이 문서를 준수하는 KM 은 "application/json" mimetype 이 자신의 선호 mimetype 이 아니더라도 이를 거부해서는 안 된다.
Metadata	32 비트 부호 없는 정수와 문자 배열을 포함하는 구조체. 정수는 문자 배열의		

Name	설명	Type	비고
	크기를 지정한다.		
Metadata_size	버퍼의 크기	32 bit unsigned integer	메타데이터 버퍼의 크기(문자 수)
Metadata_buffer	데이터 버퍼	Array of bytes (octets)	반환되는 메타데이터를 담은 버퍼
Status	요청의 성공/실패	32 bit unsigned integer	status 값의 의미는 다음과 같다. 0 성공. 1 연결은 성공했으나 피어가 연결되지 않음. 2 사용 가능한 키가 부족하여 GET_KEY 실패. 3 피어 애플리케이션이 아직 연결되지 않아 GET_KEY 실패. 4 사용 가능한 QKD 연결 없음. 5 "KSID"가 이미 사용 중이어서 OPEN_CONNECT 실패. 6 TIMEOUT_ERROR 지정된 TIMEOUT 으로 인해 호출 실패. 7 요청한 QoS 설정을 충족할 수 없어 OPEN 실패, 반환값에 대안 제안 포함. 8 메타데이터 필드 크기가 부족하여 GET_KEY 실패. 반환된 Metadata_size 값은 메타데이터에 필요한 최소 크기를 담는다. 필요한 경우 향후 다른 값이 추가될 수 있다.

역자 주 – Destination 의 비고는 원문에 "the source will integrate the URI of the client"로 적혀 있어 그대로 옮겼으며, destination 의 오기로 보인다. status 값 7 의 원문("counter proposal included in returnhas occurred.") 역시 문장이 깨져 있어 뜻이 통하도록 옮겼다.

중요 고려사항:

- 1) 하나의 애플리케이션은 서로 다른 "key stream ID"를 통해 여러 개의 독립적인 키 스트림을 관리할 수 있다(예: 송신 메시지용 키 집합과 수신 메시지용 키 집합을 각각 별도의 "Key_stream_ID"로 식별). 하나의 애플리케이션이 요청할 수 있는 "Key_stream_ID"의 수에는 제한이 없다.
- 2) Key_stream_ID 는 키 자료를 고유하게 식별하면서도 그로부터 키 자료를 유도할 수 없는 방식으로 생성해야 한다.
- 3) status 파라미터를 통해 애플리케이션으로 전달되는 오류 메시지를 제외하고, 이 QKD KM 서비스의 내부 동작을 어떻게 구현할지는 다루지 않는다. 이는 구현자에게 맡긴다.
- 4) QoS 의 priority 파라미터를 이 QKD KM 서비스의 내부 동작에서 어떻게 처리할지는 다루지 않는다. 이는 구현자에게 맡긴다.
- 5) 이 계층이 관리하는 QKD 비트 풀 내 키의 수명은 규정하지 않으며, 그 시점에 취할 동작도 규정하지 않는다. 이는 구현자에게 맡긴다.
- 6) API 는 source/destination 쌍을 URI 로 구현해야 하는 것이 필수이나, 다른 API 와의 호환성을 위해 다른 유형의 데이터도 해석할 수 있다.
- 7) 이 API 는 URI 에 일부 정보(예: OAuth 토큰)를 통합하여 인증, 인가 및 과금(AAA)을 관리할 수 있다. 다만 API 가 이 정보를 처리할 의무는 없으며, AAA 정보 처리를 담당하는 하위 시스템으로 전달할 수 있다.
- 8) metadata 필드는 키 관리 업무에 도움이 되는 정보를 제공하도록 설계되었다. 이 API 는 키 관리를 다루지 않는다.
- 9) metadata QoS 는 KM 을 호출하는 클라이언트가 제공하고 그 권한하에 두는 Metadata_size 값과 문자 블록으로 구성된다. 이 메모리 블록의 할당과 해제는 클라이언트의 책임이다. KM 은 각 GET_KEY 호출에서 이

메모리 블록의 크기를 확인해야 하며, 그 값이 부족하면 반환 시 크기 필드에 현재의 최소 필요값을 대신 넣어야 한다. 이 경우 호출 자체는 오류 값과 함께 반환되며, 클라이언트에 키를 전달해서는 안 된다.

<표 3>

Name	설명	Type	비고
"age"	해당 키 청크가 전달 가능해진 이후 경과한 시간	32 bit unsigned integer	msec 단위로 측정.
"hops"	해당 키 청크를 전달하기 위해 거친 신뢰 노드의 수	32 bit unsigned integer	0 = 직접 연결(중간에 신뢰 노드 없음).

10) KM 이 반환할 수 있는 메타데이터의 예는 다음과 같다.

- KM 벤더는 지정된 mimetype 의 형식을 지키는 범위에서 원하는 값을 추가할 수 있다.

역자 주 - 원문에서도 <표 3>이 10) 항목보다 앞에 배치되어 있다. 배치를 그대로 두었다.

6.2 QKD 애플리케이션 인터페이스의 시퀀스 다이어그램

6.2.1 일반(General)

모든 시퀀스에는 다섯 주체가 등장한다.

- 1) 호스트 A 의 애플리케이션, APPA.
- 2) 호스트 B 의 애플리케이션 APPB, 그리고 호스트 Z 의 애플리케이션 APPZ.
- 3) 호스트 A 의 QKD 키 관리자(사이트 A 와 사이트 B 를 연결하는 QKD 시스템의 모듈 A 에 있음), QKDA.
- 4) 호스트 B 의 QKD 키 관리 계층 피어(사이트 A 와 사이트 B 를 연결하는 QKD 시스템의 모듈 B 에 있음), QKDB.
- 5) QKD 네트워크에서 호스트 A 의 QKD 키 관리자(모듈 A 에 있음) QKD_SERVA 와, 호스트 Z 의 QKD 키 관리자(모듈 Z 에 있음) QKD_SERVZ.

다섯 가지 시퀀스를 제시한다.

- 1) APPA 와 APPB 가 사전 정의된 key stream ID 를 갖지 않고, 단일 링크 시나리오에서 성공적으로 수행하는 경우.
- 2) APPA 와 APPB 가 사전 정의된 key stream ID 를 갖지 않고, 단일 링크 시나리오에서 GET_KEY() 호출이 타임아웃되는 경우.
- 3) APPA 와 APPB 가 키 연관에 사용할 사전 정의된 key stream ID 를 모두 알고 있으며, 단일 링크 시나리오에서 성공적으로 수행하는 경우.
- 4) APPA 와 APPB 가 키 연관에 사용할 사전 정의된 key stream ID 를 모두 알고 있으나, 단일 링크 시나리오에서 GET_KEY() 호출이 타임아웃되는 경우.
- 5) APPA 와 APPZ 가 사전 정의된 key stream ID 를 갖지 않고, QKD 네트워크를 통해 성공적으로 수행하는 경우.

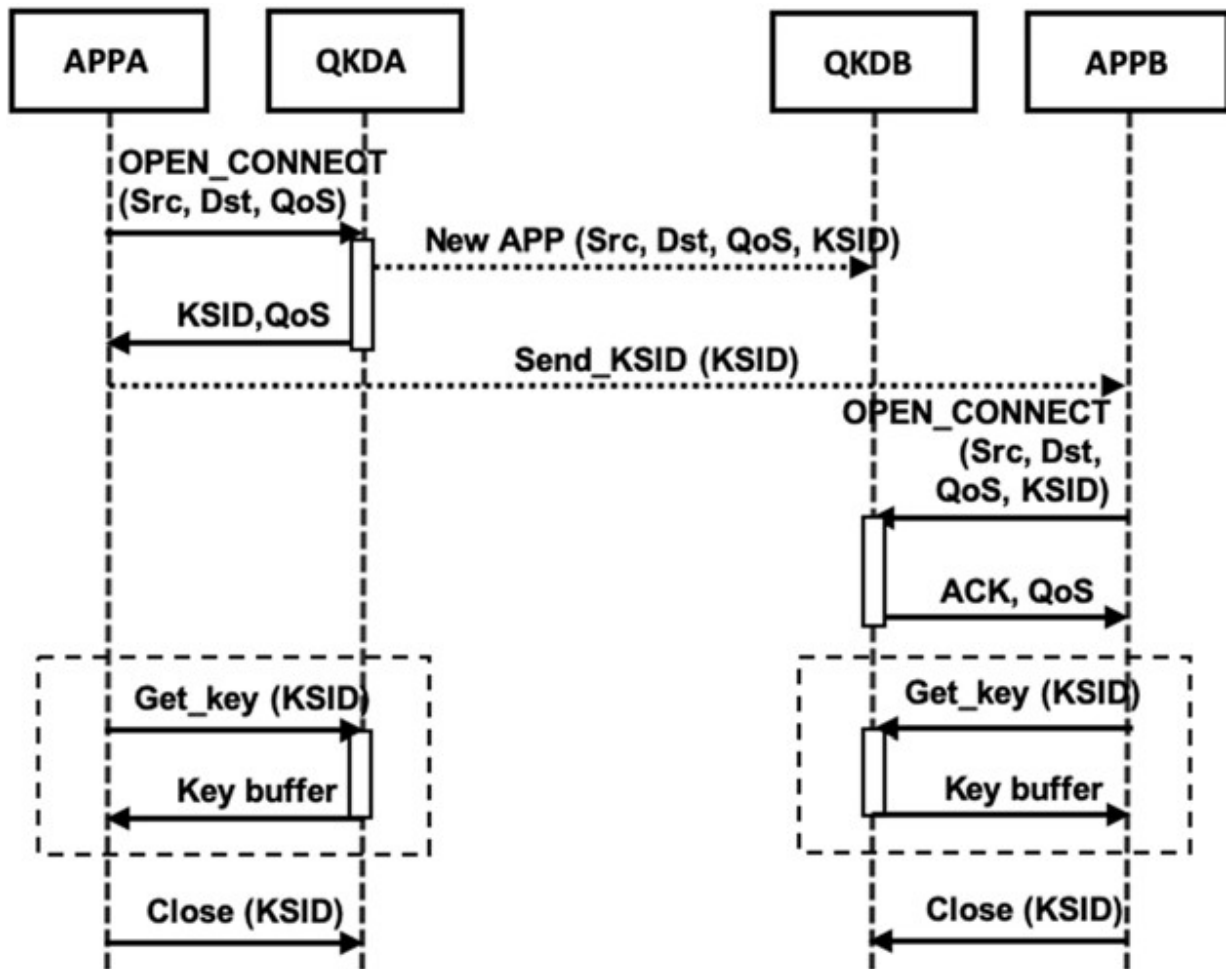
이 절에서 다루는 시나리오에서는 로컬 애플리케이션과 QKD 키 관리자 간의 통신이 사용자의 로컬 보안 경계 내에서 이루어진다고 가정할 수 있다.

6.2.2 경우 1: 단일 링크 시나리오에서 KSID 가 정의되지 않은 경우

이 경우 APPA 는 애플리케이션의 출발지, 목적지 및 원하는 QoS 를 지정하고(그리고 Key_stream_ID 파라미터에 NULL 값을 넣어) OPEN_CONNECT()를 호출하며 시퀀스를 시작한다. 단일 링크 시나리오이고 QKD 시스템이 피어를 탐색할 필요가 없으므로, OPEN_CONNECT()는 곧바로 Key_stream_ID 값과 성공을 나타내는 status 정보를 반환한다. APPA 는 이 Key_stream_ID 값을 공개 채널로 APPB 에 보낸 후 키를 모으기 시작하거나 APPB 도 연결되기를 기다린다. APPB 는 공유할 KSID 가 담긴 메시지를 APPA 로부터 공개 채널로 받기 위해 대기한다. APPB 는 KSID 를 받으면 자신의 Key_stream_ID 를 등록하기 위해 OPEN_CONNECT()를 호출해야 하며, APPA 가 이미 원하는 서비스 수준(QoS)을 지정했다면 이를 반복할 필요는 없다. APPB 는 OPEN_CONNECT 가 반환한 status 파라미터로 성공 여부를 확인하고 APPA 와 만나야 한다. 이 시점에 키 연관이 검증되어 수립되며, QKD 시스템이 정상

동작 중이면 status 는 성공으로 반환되어야 한다. 그 후 APPA 와 APPB 는 안전한 키가 더 이상 필요하지 않을 때까지 GET_KEY()를 호출하고, 이어서 CLOSE() 함수를 호출하여 Key_stream_ID 연관을 적시에 종료한다.

주 – SEND_KSID()나 New APP 은 QKD 애플리케이션 인터페이스의 범위에 속하지 않는다. 해당 경우의 기능을 설명하기 위해 포함한 것이다.

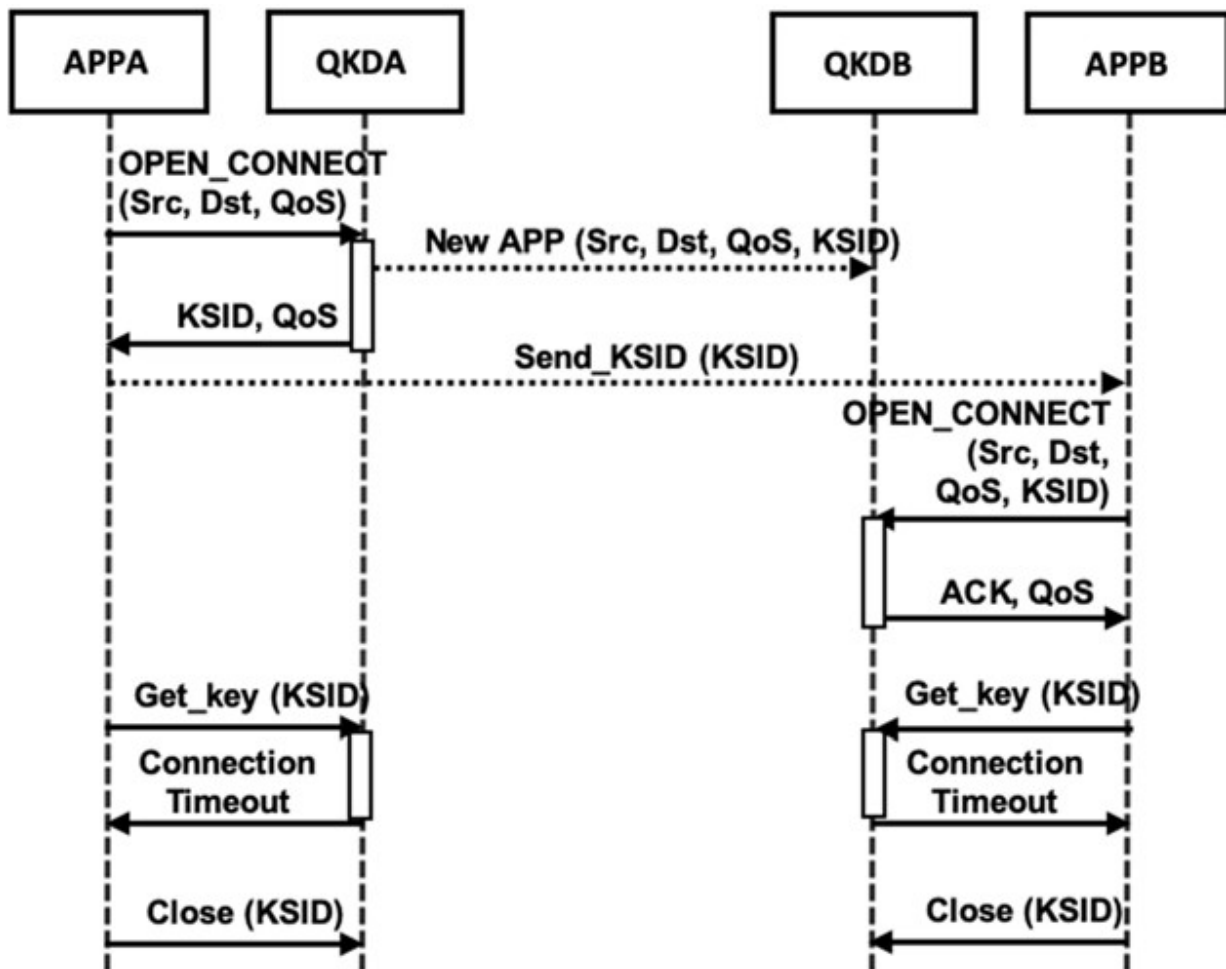


(그림 3) 경우 1 – KSID 가 정의되지 않은 경우

6.2.3 경우 2: 단일 링크 시나리오에서 KSID 가 정의되지 않고 키 획득 호출이 실패한 경우

경우 1 과 동일하나, 어떤 이유로 QKDA 가 OPEN_CONNECT()의 TIMEOUT 파라미터로 지정된 시간 내에 QKDB 와 필요한 키 정보를 교환하지 못한다. APPA 와 APPB 모두 GET_KEY() 호출의 status 파라미터로 TIMEOUT ERROR 를 받는다.

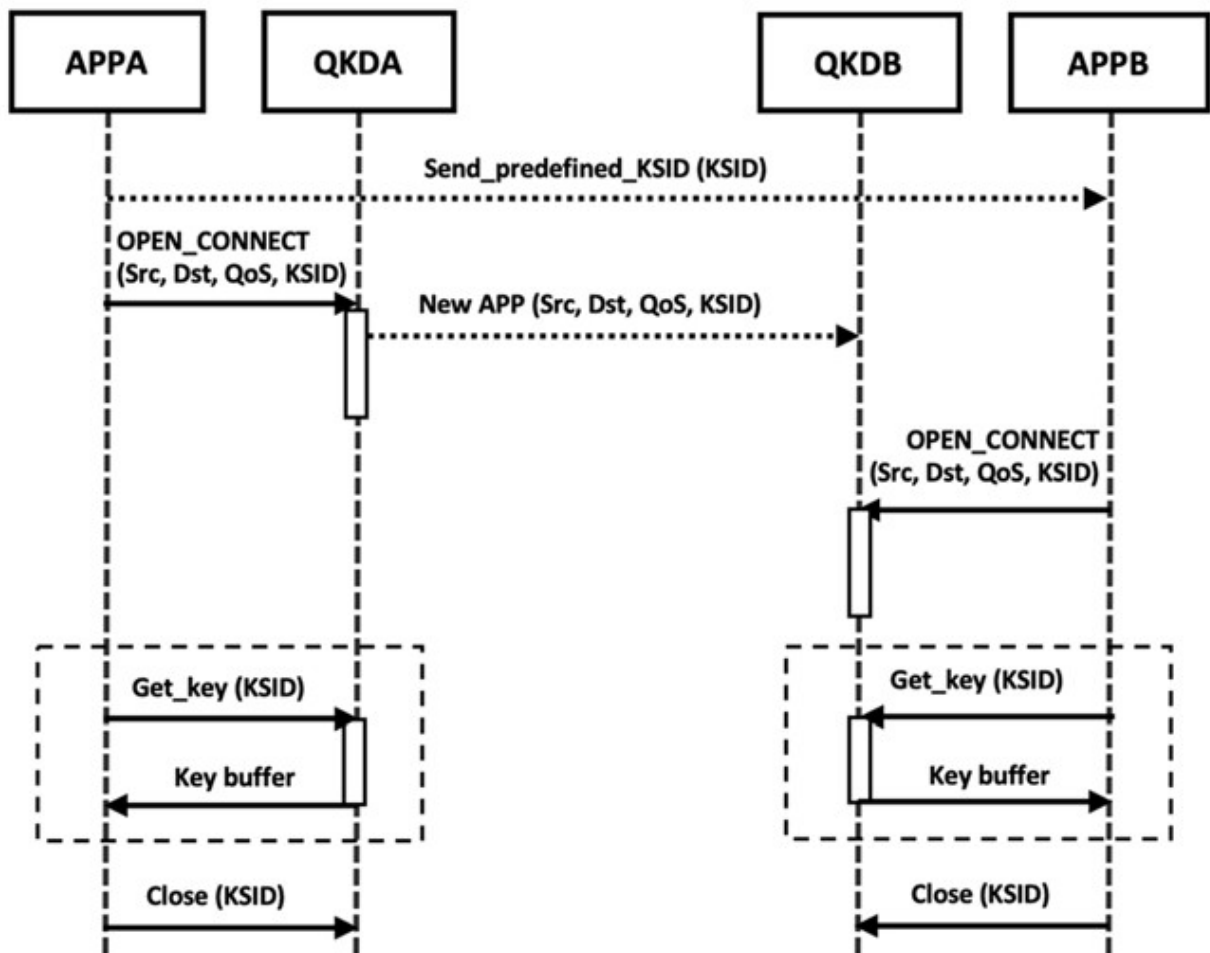
주 – SEND_KSID()나 New App 은 QKD 애플리케이션 인터페이스의 범위에 속하지 않는다. 해당 경우의 기능을 설명하기 위해 포함한 것이다.



(그림 4) 경우 2 - KSID 가 정의되지 않고 키 획득 호출이 실패한 경우

6.2.4 경우 3: 단일 링크 시나리오에서 KSID 가 사전 정의된 경우

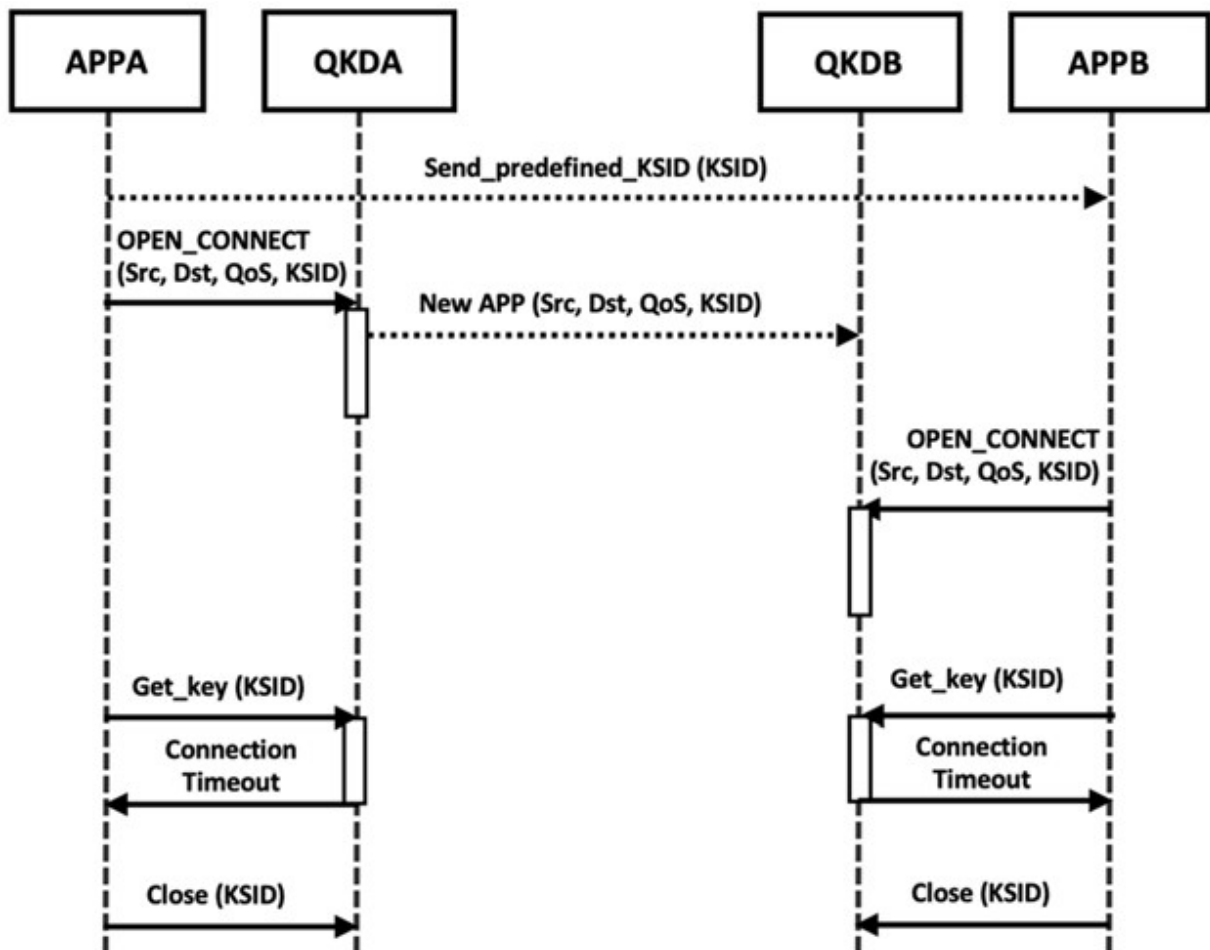
이 경우 누가 시퀀스를 시작하는지는 중요하지 않다. APPA 와 APPB 모두 원하는 서비스 수준과 사전 정의된 (최초 상호작용) 값을 Key_stream_ID 파라미터에 지정하여 OPEN_CONNECT()를 호출한다. OPEN_CONNECT()는 지정된 Key_stream_ID 가 이미 사용 중이 아님을 확인하고 성공을 나타내는 status 정보를 반환한다. OPEN_CONNECT 호출 이후 APPA 와 APPB 사이에 Key_stream_ID 정보를 주고받을 필요는 없다. QKDA 와 QKDB 는 동기화 및 검증을 위해 OPEN_CONNECT 호출 내에서 정보를 교환할 수 있다. 이 시점에 QKDA 와 QKDB 사이에 키 연관이 검증되어 수립되며, QKD 시스템이 정상 동작 중이면 status 는 성공으로 반환된다. 그 후 APPA 와 APPB 는 안전한 키가 더 이상 필요하지 않을 때까지 GET_KEY()를 호출하고, 이어서 CLOSE() 함수를 호출하여 Key_stream_ID 연관을 적시에 종료한다.



(그림 5) 경우 3 - KSID 가 사전 정의된 경우와 키 획득 호출

6.2.5 경우 4: 단일 링크 시나리오에서 KSID 가 사전 정의되고 키 획득 호출이 실패한 경우

경우 3 과 동일하나, 어떤 이유로 QKDA 가 OPEN_CONNECT()의 TIMEOUT 파라미터로 지정된 시간 내에 QKDB 와 필요한 정보를 교환하지 못한다. APPA 와 APPB 모두 GET_KEY() 호출의 status 파라미터로 TIMEOUT ERROR 를 받는다.



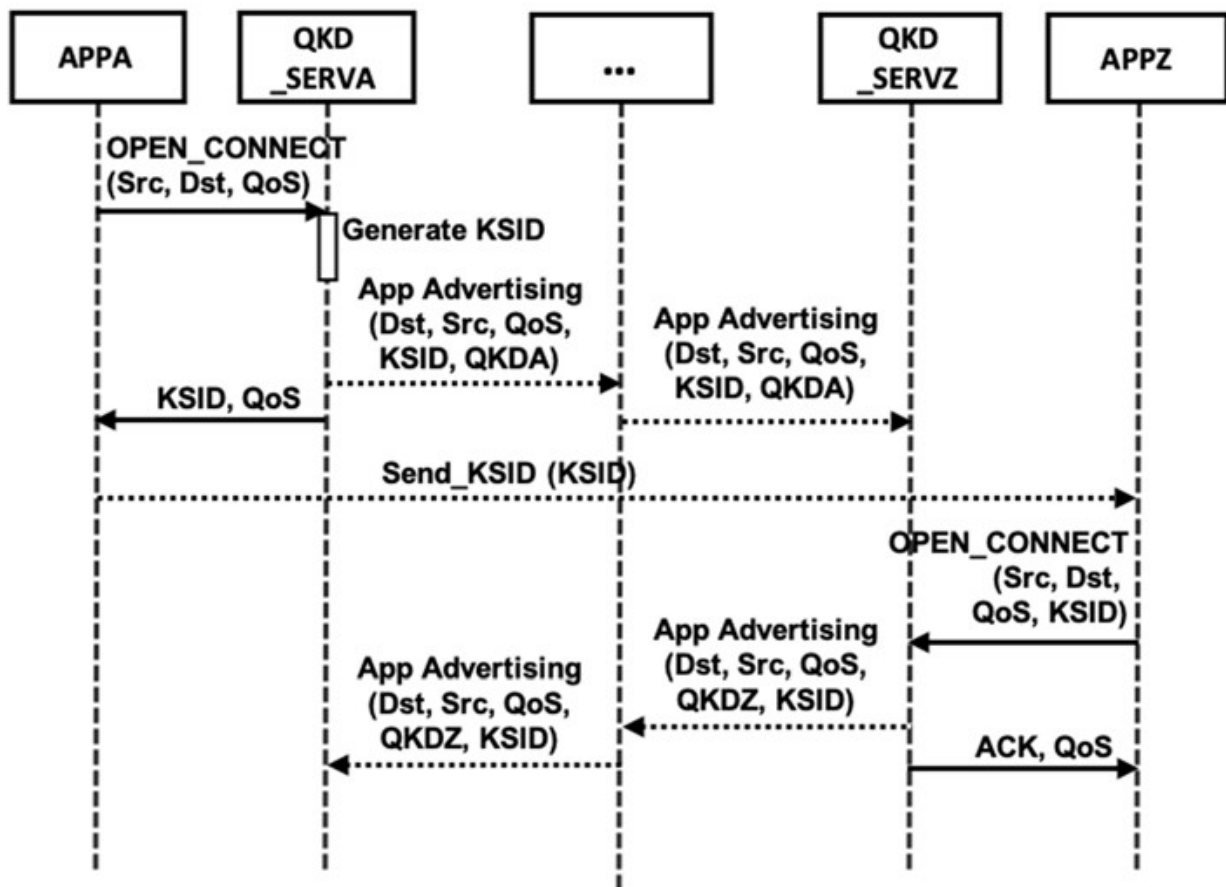
(그림 6) 경우 4 - KSID 가 사전 정의된 경우와 키 획득 호출 실패

6.2.6 경우 5: QKD 네트워크에서의 애플리케이션 탐색

네트워크 애플리케이션과의 통합은 앞에서 기술한 것과 유사한 아래의 작업 흐름을 따르며, 애플리케이션은 OPEN_CONNECT()를 통해 QKD 키 서버에 연결해야 한다. 여기에 더해, QKD 키 서버가 요청된 세션의 종단 간 피어를 찾는 중간 단계가 있다. 이는 이 문서의 범위를 벗어나며 여러 방식으로 이루어질 수 있다. 예를 들어 애플리케이션 탐색을 위해 QKD 사이트 간에 시그널링을 하거나, 중간의 중앙 저장소 또는 관리 애플리케이션(예: 중앙 키 관리 시스템이나 소프트웨어 정의 네트워킹(SDN) 제어기)에 통지하는 방식이 있다. 또한 키 릴레이를 위해 QKD 키 서버 간의 모든 통신 채널에는 정보 이론적으로 안전한(ITS) 메커니즘을 통합해야 하나, 이러한 보안 메커니즘을 정의하는 것은 이 문서의 범위를 벗어난다.

이 경우는 단일 점대점 QKD 링크가 아니라 QKD 네트워크가 구축된 상황을 다룬다. 애플리케이션 APPA가 애플리케이션 APPZ와 일련의 공유 비밀 비트를 얻고자 한다. 위치 A와 위치 Z의 QKD 모듈 사이에는 직접 링크가 없으므로, 요청에 따라 APPZ의 QKD 종단점을 식별할 수 있는 지원 설비를 시스템이 제공해야 한다. (그림 7)에서는 이러한 설비를 QKD_SERVA와 QKD_SERVZ로 표시하였다. APPA가 QKD_SERVA에 OPEN_CONNECT()를 보낼 때, QKD_SERVA 역시 APPZ가 어느 QKD 종단점에 연결되어 있는지 알지 못한다. 가장 먼저 수행할 동작은 APPA가 생성하려는 세션에 대한 KSID의 생성이다. 그런 다음 QKD_SERVA는 출발지(APPA), 목적지(APPZ), 생성된 KSID, 로컬 QKD 모듈(QKDA 또는 QKD_SERVA) 및 QoS 정보를 알리며 이 새로운 애플리케이션 요청을 광고해야 한다. 이 광고 과정은 새로운 애플리케이션과 그 요구사항을 QKD 네트워크 전체에 브로드캐스트하는 전통적인 라우팅 방식으로 수행하거나, 이러한 통지를 담당하고 QKD 모듈 피어에 적절한 정보를 제공하는 중앙집중형 관리 시스템을 통해 수행할 수 있다. 이 정보가 전달되면 QKD_SERVA는 생성된 KSID를 APPA에 보낸다. 이후 APPA는 QKD_SERVA가 반환한 KSID를 피어인 APPZ에 알려야 한다. APPZ가 자신의 QKD 종단점(QKD_SERVZ)에 연결하면 동일한 상황이 벌어지는데, 차이가 있다면 (이 구현에서는) 중간의 광고 절차 덕분에 QKD_SERVZ가 QKD_SERVA에 연결된 애플리케이션을 이미 알고 있고, APPZ도 APPA가 앞서 공유한 유효한 KSID를 알고 있다는

점이다. 이 작업 흐름은 특정 구현에 대한 예시이며, 구조적 결정(예: 중앙집중형 관리 시스템의 존재 여부)에 따라 달라질 수 있다.



(그림 7) 경우 5 - QKD 네트워크에서의 애플리케이션 탐색 작업 흐름

애플리케이션은 전역적으로 고유하기만 하다면 사용할 KSID 를 사전에 정의할 수도, 하지 않을 수도 있다. 연결 타임아웃과 관련된 문제는 로컬에서 관리되며, 각각 6.2.3 절과 6.2.5 절의 경우 2 및 경우 4 와 유사하게 오류로 처리된다.

부속서 A:
공란(Void)

부속서 B (정보성):

기존 키 관리 시스템

QKD 시스템 내에서 사용하기 위해 기존 키 관리 시스템을 조사하였다. 그러한 프로토콜 중 하나로 OASIS 개방형 표준 컨소시엄(<https://www.oasis-open.org>)의 KMIP 이니셔티브를 검토하였다. KMIP은 클라이언트와 키 서버 간 통신을 위한 프로토콜과 인터페이스를 규정한다. 반면 QKD는 분산 패러다임으로 동작하여, 키가 이미 QKD 종단점(QKD 모듈)에 분배되어 있고, 이를 독립적인 동기화 스트림으로 역다중화하여 필요에 따라 해당 통신 종단점의 여러 애플리케이션에 분배한다.

QKD 시스템에도 KMIP 프레임워크를 채택하는 방안과, 다음과 같이 QKD를 사용하여 KMIP을 강화하는 방안을 검토하였다.

- 1) KMIP 인증 프로토콜의 보안 강화. KMIP은 키 관리 시스템과 암호 클라이언트가 서로를 식별하는 메커니즘을 프로토콜의 일부로 규정하지 않는다. 대신 KMIP은 이러한 식별을 어떻게 수립할지 규정하는 기존의 상호 인증 표준에 의존한다. KMIP은 현재 두 가지 인증 프로파일을 정의하는데, 첫 번째는 TLS 기반이고 두 번째는 HTTPS 기반이다. 두 프로파일 모두에서 클라이언트와 서버는 KMIP 요청 및 응답의 참여자로서 자신을 식별하기 위해 디지털 인증서를 사용한다. 기업 키 관리자가 암호 클라이언트의 신원을 알게 되는 등록 메커니즘은 KMIP에 정의되어 있지 않다. 암호 클라이언트가 자신을 식별하는 데 사용하는 자격 증명은 키 관리 시스템의 요청 처리를 단순화하기 위해 요청 메시지의 일부로 프로토콜에 포함될 수 있다. 그러나 자격 증명 요소는 인증됨이 보장되지 않으므로 인증 용도로 사용하기 위한 것은 아니다. KMIP 교환의 메시지 무결성과 개체 인증은 TLS가 제공한다. KMIP 메시지의 보안을 강화하는 데 사용할 수 있는 다른 메커니즘(예: TLS나 HTTPS 키 교환 과정에 QKD를 포함하는 방식)은 현재 KMIP에 정의되어 있지 않다.

- 2) KMIP 메시지 교환에 QKD 일회용 패드 터널을 사용하여 키 관리 시스템의 보안을 강화.

KMIP 서버를 위한 제 3의 키 공급원으로 QKD 키를 포함하는 방안.

Key Management Interoperability Protocol Specification Version 1.4는 OASIS 표준이다 [i.1].

부속서 C (정보성):

이 API 와 ETSI GS QKD 014 "REST 기반 키 전달 API 의 프로토콜 및 데이터 형식"의 관계

이 API 는 높은 유연성을 유지하면서도 가능한 한 작게 설계되었다. 기본 설계 방향은, 연결된 둘 이상의 원격 애플리케이션으로부터 최소한의 오버헤드로 키 처리율을 극대화할 수 있도록 연속적인 키 스트림을 생성할 수 있는 API 를 만드는 것이다. 이는 또한 이 인터페이스가 애플리케이션과 하위 QKD 네트워크 사이에서 특정 서비스 품질 (QoS)을 연관시키고 협상할 수 있는 세션을 제공할 수 있음을 의미한다. 네트워크는 QoS 요청에 기반하여 모든 요청을 조직해 QKD 기반 서비스의 이용을 극대화할 수 있다. 예를 들어 우선순위를 할당하고, 향후 요청을 위한 키 프로비저닝과 예약을 조직할 수 있다. 이 문서는 QoS 와 함께 네트워크 메타데이터의 관리도 도입하는데, 이는 네트워크 통계나 QKD 장치의 디버그 정보와 같이 네트워크에 대한 지식을 얻는 데 유용하다.

이 API 는 또한 구현 방식에 구애받지 않는다. 구현에 필요한 프로그래밍 언어, 라이브러리, 기타 도구와 독립적이며, 서비스 요구사항을 더 잘 충족하도록 그 선택을 구현자에게 맡긴다. 즉, 구체적인 구현의 필요에 맞게 프로토콜 스택을 선택할 수 있다. 자원이 제한된 장치를 위한 부담이 적은 작은 스택, 파싱과 검사가 쉬워 보안 평가 및 인증이 용이한 (대개 작은) 구현을 위한 프로토콜, 또는 다른 애플리케이션과의 통합을 쉽게 하는 복잡한 프로토콜 스택(Corba, REST over HTTP(S) 등)을 선택할 수 있다. 단순한 원시 소켓부터 SSL/TLS 터널, ZeroMQ/Protobuf 와 같은 보다 안전한 통신 방식까지 사용하여 이 API 를 이용할 수 있다.

URI 를 사용함으로써 다양한 인증(예: PAP, Kerberos, OAuth 등), 인가 및 과금(AAA) 프로토콜의 사용도 가능해진다. 또한 URI 를 사용하면 임의의 애플리케이션 쌍에 대해 다중 홉 (키 릴레이 포함) 연결을 비롯한 다양한 라우팅 프로토콜을 사용할 수 있다.

이에 비해 014 키 전달 API 는 완전히 다른 설계 원칙을 따른다(014 와 004 API 의 관계에 대한 추가 정보는 ETSI GS QKD 014 [i.2]의 부속서 A 에서 찾을 수 있다). ETSI GS QKD 014 [i.2]는 단일 키 요청 API 로 구상되었으며, 애플리케이션은 항상 REST 요청을 사용해 키를 개별적으로 요청해야 하고(키 스트림 형태가 아님), 키 식별자의 전송과 처리를 스스로 책임진다. 또한 세션 개념이 없다.

ETSI GS QKD 014 [i.2] API 는 HTTPS 프로토콜과 JSON 데이터 형식의 사용을 규정한다. 이는 구현을 쉽게 하지만, HTTPS 와 JSON 이 구현에 상위 수준의 라이브러리나 프레임워크를 요구하므로 일부 제약도 따른다. 이에 비해 004 API 는 어떠한 라이브러리도 없이 매우 간결하게, 그리고 전적으로 표준 C 언어로 구현할 수 있어 보안 분석이 단순해진다.

ETSI GS QKD 014 [i.2] API 는 상위 수준 도구(웹 기반)로 구축되고 004 API 의 QoS 가 없는 단순화된 형태로 볼 수 있다. ETSI GS QKD 014 [i.2] API 는 이 문서의 API 위에 구현할 수 있으나, QoS 는 기본 상수로 설정해야 한다. 기본적인 접근은, REST 요청을 처리하는 웹 서버가 시작 시 이 문서 API 의 OPEN_CONNECT() 호출을, 중지 시 대응하는 API CLOSE() 호출을 수행하는 것이다. 키와 관련된 각 REST 요청에는 이 문서 API 의 GET_KEY() 호출 한 번이 필요하다. 이러한 근사는 모든 상위 수준 처리를 웹 측에, 모든 키 관리를 이 문서의 API 에 맡긴다. 반대 방향은 (추가적인 동기화 없이는) 항상 가능하지는 않다. 이 문서의 세션 개념은 API 사용자가 카운터만 입력으로 하는 단순한 요청으로 여러 대응 키를 얻을 수 있게 하는 반면, ETSI GS QKD 014 [i.2]에서는 각 키마다 새로운 고유 키 식별자가 부여되고 사용자가 이를 전달해야 하기 때문이다.

부속서 D (정보성):

참고 자료(Bibliography)

역자 주 – 참고 자료 항목은 문헌의 서지 정보이므로 원문 그대로 두었다.

- C.H. Bennett and G. Brassard: "Quantum Cryptography: Public Key Distribution and Coin Tossing", Proceedings of IEEE International Conference on Computers Systems and Signal Processing, Bangalore India, December 1984, pp 175-179. (<http://www.research.ibm.com/people/b/bennetc/bennettc198469790513.pdf>)
- Nicolas Gisin, Grégoire Ribordy, Wolfgang Tittel and Hugo Zbinden: "Quantum cryptography, Reviews of Modern Physics", vol 74, 145-195 (2002).
- Valerio Scarani, Helle Bechmann-Pasquinucci, Nicolas J. Cerf, Miloslav Dušek, Norbert Lütkenhaus and Momtchil Peev: "The security of practical quantum key distribution", vol. 81, 1301-1351 (2009). (<http://arxiv.org/abs/0802.4155>)
- D. Gottesman, H.-K. Lo, N. Lütkenhaus and J. Preskill: "Security of quantum key distribution with imperfect devices", vol. 5, 325-360 (2004). (<http://arxiv.org/abs/quant-ph/0212066>)
- "White Paper on Quantum Key Distribution and Cryptography", Preprint arXiv:quant-ph/0701168, Alléaume R, Bouda J, Branciard C, Debuisschert T, Dianati M, Gisin N, Godfrey M, Grangier Ph, Länger T, Leverrier A, Lütkenhaus N, Painchault P, Peev M, Poppe A, Pornin Th, Rarity J, Renner R, Ribordy G, Rigidel M, Salvail L, Shields A, Weinfurter H, Zeilinger, A, 2006 SECOQC.
- UQC Report: "Updating Quantum Cryptography", Quantum Physics (quant-ph); Cryptography and Security. Donna Dodson, Mikio Fujiwara, Philippe Grangier, Masahito Hayashi, Kentaro Imafuku, Ken-ichi Kitayama, Prem Kumar, Christian Kurtsiefer, Gaby Lenhart, Norbert Luetkenhaus, Tsutomu Matsumoto, William J. Munro, Tsuyoshi Nishioka, Momtchil Peev, Masahide Sasaki, Yutaka Sata, Atsushi Takada, Masahiro Takeoka, Kiyoshi Tamaki, Hidema Tanaka, Yasuhiro Tokura, Akihisa Tomita, Morio Toyoshima, Rodney van Meter, Atsuhiko Yamagishi, Yoshihisa Yamamoto and Akihiro Yamamura, 2009. (<http://arxiv.org/abs/0905.4325>)
- "Introduction to Quantum Key distribution", V. Martin, J. Martínez-Mateo and M. Peev, Wiley Encyclopedia of Electrical and Electronics Engineering, 2017. (10.1002/047134608X.W8354)
- Menezes A. J., van Oorschot P. C. and Vanstone S. A.: "Handbook of Applied Cryptography", (Boca Raton: CRC Press) 1997.
- Schneier B.: "Applied Cryptography", 1996, (New York: John Wiley).
- NIST Special Publication 800-53 Revision 4 "Security and Privacy Controls for Federal Information Systems and Organizations", Joint Task Force transformation initiative. (<http://dx.doi.org/10.6028/NIST.SP.800-53r4> April 2013)
- OMG IDL Syntax and Semantics, Object Management Group Inc., formal/02-06-39 (CORBA 3.0 - OMG IDL Syntax and Semantics chapter). (<http://www.omg.org/cgi-bin/doc?formal/02-06-39>)

부속서 E (정보성):

변경 이력

일자	버전	변경 정보
2010 년 12 월	1.1.1	발행.
2018 년 5 월	1.2.0	인터페이스 간소화, QoS 추가, URI 주소 지정, 키 스트림에 대한 UUID 식별자.
2018 년 5 월	1.2.1	1.2.0 에 대한 사소한 수정(버전 번호, 이력 설명, 기여자 표기 및 서식 갱신).
2019 년 1 월	1.2.2	추가적인 사소한 수정. 부속서 C 명확화 및 오래된 그림 삭제.
2019 년 1 월	1.2.3	(그림 1)과 (그림 2)를 포함하여 본문 전반의 용어 명확화. 부속서 B 와 부속서 D 개정.
2019 년 2 월	1.2.4	추가 명확화. Clean Code 규칙에 따른 호출의 일관된 명명 규칙. 그림 명칭 오류 수정. 경우 5 논의 및 재작성, (그림 6) 수정 및 사소한 수정.
2019 년 9 월	1.2.5	(그림 2)와 (그림 3)의 오류 수정. 본문 명확화. QoS 를 inout 인자로 정의하고 그 동작을 명확히 하였으며, 이를 반영하여 작업 흐름 그림 수정. 오류 코드 추가.
2019 년 11 월	1.2.9	URI 를 넘어서는 확장 허용에 대한 설명. 키에 메타데이터를 추가하기 위한 구조.
2020 년 1 월	1.3.1	키 관리에 대한 논의 축소. 네트워크 사례를 부록에서 본문으로 이동. 사소한 오류 수정. 메타데이터 유형 수정.
2020 년 2 월	1.3.3	메타데이터 필드 수정 및 정리.
2020 년 3 월	1.3.4	부속서 C.
2020 년 5 월	1.3.5	부속서 C 에 대한 논의 수집. 용어 정의 절 개정.
2020 년 5 월	1.3.6	누락된 약어 추가. 웹 주소 오류 수정.

이력(History)

문서 이력

V1.1.1	2010 년 12 월	발행
V2.1.1	2020 년 8 월	발행